

Hợp nhất tập hợp — DSU (Disjoint Set Union)

PHỤ THUỘC A/03 cho phân tích khâu hao; A/04 cho bất biến. Là điều kiện cần cho C/15 (Kruskal) và nhiều bài ở C/17.

TÓM TẮT

DSU trả lời đúng hai câu hỏi: “hai phần tử này có cùng nhóm không” và “gộp hai nhóm lại”. Chỉ hai thao tác, khoảng hai chục dòng code, nhưng đó là cấu trúc có tỉ lệ sức mạnh trên độ dài code cao nhất trong cả quyển sách — nó đứng sau thuật toán Kruskal, các bài liên thông xử lý ngoại tuyến, kỹ thuật gộp nhỏ vào lớn, và nhiều mẹo đếm. Hai tối ưu đơn giản — gộp theo kích thước và nén đường — đưa chi phí xuống gần như hằng số, với một phân tích khâu hao thuộc loại đẹp nhất ngành. Điểm yếu duy nhất, nhưng rất quan trọng: DSU *không xoá được*. Nếu chỉ nhớ một điều: DSU là cấu trúc một chiều — nhóm chỉ có thể to ra — và mọi kỹ thuật nâng cao trong chương chỉ là cách né hạn chế đó.

KIẾN THỨC CẦN CÓ TRƯỚC

- **quan hệ tương đương** — quan hệ có tính phản xạ, đối xứng, bắc cầu; nó chia tập thành các lớp rời nhau. DSU chính là cấu trúc duy trì các lớp đó khi quan hệ được bổ sung dần.
- **đại diện (representative)** — một phần tử được chọn làm “tên” của cả nhóm; hai phần tử cùng nhóm khi và chỉ khi có cùng đại diện.
- **gộp theo kích thước / hạng** — luôn treo cây nhỏ vào gốc cây lớn, để chiều cao không vượt $O(\log n)$.
- **nén đường (path compression)** — sau mỗi lần tìm đại diện, nối thẳng mọi nút trên đường lên gốc, làm các lần tìm sau rẻ hơn.
- **hàm Ackermann ngược $\alpha(n)$** — hàm tăng cực chậm; với mọi n trong vũ trụ thực tế thì $\alpha(n) \leq 4$. Chi phí khâu hao của DSU là $O(\alpha(n))$.
- **gộp nhỏ vào lớn (small to large)** — kỹ thuật tổng quát hơn: khi gộp hai tập hợp thật sự, luôn duyệt tập nhỏ và đổ vào tập lớn; tổng chi phí $O(n \log n)$ vì mỗi phần tử chỉ bị chuyển $O(\log n)$ lần.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao chỉ cần hai mẹo đơn giản — gộp theo kích thước và nén đường — đã đưa chi phí từ $O(n)$ xuống gần như $O(1)$?
- Q2. Vì sao DSU không hỗ trợ thao tác tách nhóm, và trong bài toán bắt buộc phải tách thì làm thế nào?
- Q3. Kỹ thuật “gộp nhỏ vào lớn” cho $O(n \log n)$ bằng một lập luận rất ngắn. Lập luận đó là gì?
- Q4. DSU chỉ biết “cùng nhóm hay không”. Làm sao nó trả lời được câu hỏi tinh vi hơn như “hai phần tử này khác phe nhau”?
- Q5. Trong bài toán mà các cạnh bị xoá dần theo thời gian, vì sao xử lý ngược thời gian lại cứu được tình hình?

Có một dạng bài toán rất phổ biến: các phần tử dần dần được nối lại với nhau, và ta cần biết ai đang cùng nhóm với ai. Máy tính trong mạng được nối thêm dây; các pixel cùng màu được gộp

thành vùng; các đỉnh của đồ thị được nối thêm cạnh. Nếu mỗi lần hỏi ta lại chạy một lượt duyệt đồ thị thì chi phí là $O(n)$ cho mỗi câu hỏi — quá đắt khi có hàng trăm nghìn câu.

DSU là câu trả lời, và điều làm nó đặc biệt không phải sự tinh vi mà là sự *tiết chế*. Nó không lưu danh sách thành viên của mỗi nhóm, không lưu cấu trúc của các liên kết, không trả lời được “nhóm này gồm những ai”. Nó chỉ lưu đúng một con trỏ cha cho mỗi phần tử, và chỉ trả lời đúng câu “ai là đại diện của nhóm chứa phần tử này”. Chính vì hứa ít nên nó rẻ — lại một lần nữa nguyên tắc xuyên suốt phần B.

1.1 Từ ngây thơ tới gần như hằng số

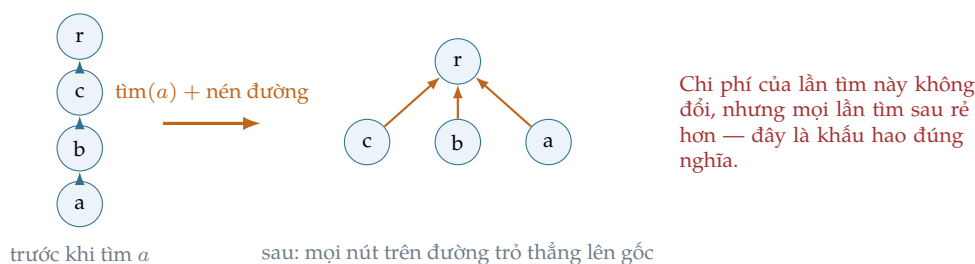
Biểu diễn: mỗi phần tử có một con trỏ cha; đi ngược theo cha tới khi gặp nút trỏ vào chính nó, đó là đại diện. Gộp hai nhóm: cho gốc này trỏ vào gốc kia.

Cách ngây thơ này có thể tạo ra một chuỗi dài n , và mỗi lần tìm tốn $O(n)$. Hai tối ưu chứa được điều đó, và điều đáng chú ý là chúng độc lập nhau nhưng bổ sung nhau.

Gộp theo kích thước. Luôn treo cây có ít phần tử hơn vào gốc của cây lớn hơn. Bất biến sinh ra: một cây có chiều cao h phải chứa ít nhất 2^h phần tử — chứng minh bằng quy nạp, vì chiều cao chỉ tăng khi hai cây cùng chiều cao gộp lại, và khi đó kích thước ít nhất gấp đôi. Suy ra chiều cao tối đa $\log_2 n$, và mọi thao tác là $O(\log n)$.

Nén đường. Trong lúc đi tìm đại diện, ta đã đi qua cả một đường; hãy nối thẳng mọi nút trên đường đó lên gốc. Việc này không tốn thêm chi phí tiệm cận nào ở lần tìm hiện tại, nhưng làm mọi lần tìm sau rẻ đi. Đây là một ví dụ mẫu mực cho ý tưởng khấu hao ở chương A/03: trả thêm một chút bây giờ, hưởng lợi nhiều lần về sau.

Dùng cả hai, chi phí khấu hao mỗi thao tác là $O(\alpha(n))$ với α là hàm Ackermann ngược — một hàm tăng chậm tới mức với n nhỏ hơn số nguyên tử trong vũ trụ quan sát được thì $\alpha(n) \leq 4$. Kết quả này do Tarjan chứng minh năm 1975 [1], và nó nổi tiếng vì kết luận rất bất ngờ: một cấu trúc chỉ dùng một mảng số nguyên lại có chi phí không phải hằng số nhưng gần tới mức không phân biệt được. Về sau người ta còn chứng minh được rằng $O(\alpha(n))$ là *tối ưu* — không cấu trúc nào làm tốt hơn cho bài toán này.



Hình 1.1. Nén đường. Điều đáng nhớ là nó không làm lần tìm hiện tại nhanh hơn — ta vẫn phải đi hết đường — mà làm cấu trúc phẳng đi cho tương lai. Kết hợp với gộp theo kích thước, chi phí khấu hao trở thành $O(\alpha(n))$, thực tế là hằng số.

1.2 Hạn chế cốt lõi: không xóa được

DSU là cấu trúc *một chiều*: nhóm chỉ có thể to ra, không bao giờ nhỏ đi. Lý do rất trực tiếp: nén đường đã phá hủy thông tin về cấu trúc gốc, nên không có cách nào biết phải tách ở đâu.

Đây không phải khiếm khuyết cài đặt mà là đánh đổi cố ý — chính việc vứt bỏ thông tin làm cấu trúc rẻ như vậy. Nhưng rất nhiều bài toán thật lại có cả xoá: cạnh mạng bị đứt, kết nối bị huỷ. Có ba cách né, và cả ba đều đáng biết vì chúng là mẫu tư duy dùng lại được.

Cách thứ nhất — đảo ngược thời gian. Nếu bài toán chỉ có xoá chứ không có thêm, hãy xử lý các truy vấn theo thứ tự ngược: xoá theo chiều xuôi trở thành thêm theo chiều ngược. Đây là một mẹo đơn giản tới mức dễ bị bỏ qua, nhưng nó giải quyết trọn vẹn cả một họ bài. Điều kiện: truy vấn phải biết trước, tức là bài toán ngoại tuyến.

Cách thứ hai — DSU có hoàn tác. Bỏ nén đường, chỉ giữ gộp theo kích thước (chi phí thành $O(\log n)$), và ghi lại mọi thay đổi vào một ngăn xếp để có thể hoàn tác. Kết hợp với chia để trị trên trục thời gian, ta giải được bài liên thông động ngoại tuyến tổng quát: mỗi cạnh tồn tại trong một khoảng thời gian, và ta duyệt cây phân đoạn trên thời gian, thêm cạnh khi đi xuống và hoàn tác khi đi lên.

Cách thứ ba — cấu trúc khác hẳn. Nếu bài buộc phải trực tuyến và có cả thêm lẫn xoá, DSU không đủ và cần cấu trúc như link-cut tree. Xếp vào loại biết là có.

CẠM BÃY

Sự kết hợp giữa nén đường và hoàn tác là không tương thích — đây là lỗi thường gặp khi người ta cố ghép hai kỹ thuật. Nén đường thay đổi nhiều con trỏ trong một lần tìm, và những thay đổi đó không được ghi lại đúng cách nếu bạn chỉ ghi các thao tác gộp. Muốn hoàn tác thì phải bỏ nén đường và chấp nhận $O(\log n)$.

1.3 Tăng cường DSU: đếm, phe phái, khoảng cách

Giống như cây tìm kiếm ở chương B/09, DSU mạnh lên rất nhiều khi được gắn thêm thông tin — và điều kiện cũng tương tự: thông tin phải *gộp được* khi hai nhóm hợp nhất.

Đếm. Lưu kích thước mỗi nhóm ở gốc: gộp thì cộng lại. Cho phép trả lời “nhóm chứa x có bao nhiêu phần tử” và “hiện có bao nhiêu nhóm”. Đây là dạng dùng nhiều nhất.

Phe phái, hay DSU có trọng số. Lưu thêm khoảng cách tương đối từ mỗi nút tới cha nó theo một nhóm cộng — thường là tính chẵn lẻ. Khi đó DSU trả lời được “ x và y cùng phe hay khác phe”, và phát hiện được mâu thuẫn khi có ràng buộc trái ngược. Ứng dụng: kiểm tra đồ thị hai phía trực tuyến, và các bài ràng buộc kiểu “ x và y khác loại”. Chú ý khi nén đường phải cộng dồn trọng số dọc đường — đây là chỗ dễ sai nhất.

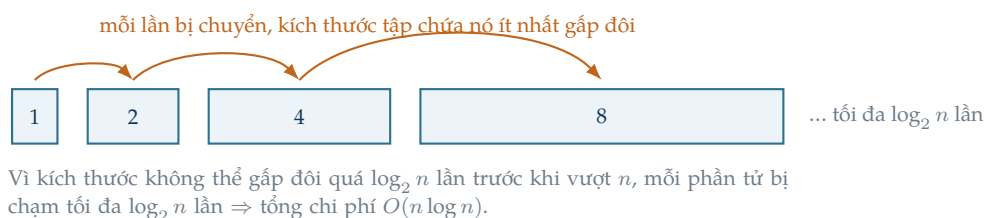
DSU trên miền giá trị — mẹo “vị trí trống kế tiếp”. Một cách dùng rất khéo: cho DSU trỏ mỗi vị trí đã bị chiếm sang vị trí trống kế tiếp bên phải. Khi cần “tìm ô trống đầu tiên từ i trở đi”, chỉ việc tìm đại diện; khi chiếm ô đó thì gộp nó với ô kế tiếp. Kỹ thuật này biến nhiều bài “tô màu đoạn, mỗi ô chỉ tô một lần” từ $O(n^2)$ thành gần tuyến tính — và nó là ví dụ điển hình cho việc dùng DSU ở nơi mà bài toán không hề nói tới “nhóm” nào cả.

1.4 Gộp nhỏ vào lớn: người anh em tổng quát hơn

DSU không lưu danh sách thành viên. Khi bài toán *cần* danh sách — ví dụ phải duyệt mọi phần tử của nhóm khi gộp — thì ta dùng kỹ thuật gộp nhỏ vào lớn: giữ tập hợp thật cho mỗi nhóm, và khi gộp thì luôn duyệt tập nhỏ hơn rồi đổ vào tập lớn hơn.

Chi phí tổng cộng là $O(n \log n)$, và lập luận cực ngắn: mỗi lần một phần tử bị chuyển, nó chuyển từ tập nhỏ sang tập lớn, nên kích thước tập chứa nó ít nhất gấp đôi. Kích thước không thể gấp đôi quá $\log_2 n$ lần, nên mỗi phần tử bị chuyển tối đa $\log_2 n$ lần. Đây là một trong những lập luận khẩu hao đẹp và dễ nhớ nhất trong quyển sách, và nó dùng lại ở rất nhiều nơi.

Biến thể quan trọng là *DSU trên cây*: khi giải các bài truy vấn về cây con — “trong cây con của u có bao nhiêu màu phân biệt” — ta duyệt cây, giữ lại cấu trúc đếm của con nặng nhất và gộp các con nhẹ vào. Chi phí $O(n \log n)$ nhờ đúng lập luận trên.



Hình 1.2. Lập luận “gộp nhỏ vào lớn”. Điều đáng học không phải kết quả mà là cách đếm: ta không tính chi phí theo từng phép gộp — có phép gộp rất đắt — mà theo *tổng số lần các phần tử bị chạm*. Cùng kiểu lập luận với khẩu hao ở A/03.

1.5 “Tối ưu” nghĩa là gì ở đây: hai cận dưới cho hợp nhất tập hợp

Mục 1 kết thúc bằng một câu đáng ngờ nếu đọc kỹ: $O(\alpha(n))$ là *tối ưu*. Tối ưu so với cái gì? Không ai chứng minh được cận dưới cho “mọi thuật toán khả dĩ” — chương D/24 giải thích vì sao điều đó gần như bất khả thi. Cái người ta chứng minh được là hai phát biểu hẹp hơn nhưng mạnh một cách bất ngờ, và chúng đáng biết vì chúng cho thấy hàm α không phải vết tích của một phép phân tích vụng về, mà là một tính chất của chính bài toán.

Kết quả thứ nhất là của Tarjan[2]. Ông xét một lớp thuật toán được mô tả rõ: những thuật toán chỉ thao tác với các phần tử qua con trỏ, đúng như cách DSU đi từ nút lên cha, không được phép làm số học trên nhãn để đi tắt. Trong lớp đó, mọi thuật toán trả lời trực tuyến m thao tác trên n phần tử đều cần $\Omega(m \alpha(m, n))$ trong trường hợp xấu nhất. Vậy trong thế giới con trỏ, nén đường cộng gộp theo kích thước không chỉ tốt: nó chạm cận.

Kết quả thứ hai mạnh hơn nhiều vì nó bỏ luôn giả thiết con trỏ. Fredman và Saks[3] làm việc trong mô hình *cell-probe*, ở đó ta chỉ đếm số ô nhớ mà thuật toán chạm vào và cho phép nó tính bất cứ thứ gì nó muốn trên các ô ấy — kể cả các mẹo bit và song song mức từ máy. Ngay trong mô hình rộng rãi đó, chi phí khẩu hao mỗi thao tác vẫn là $\Omega(\alpha)$. Nói cách khác, không có mẹo cài đặt nào đưa bài toán này xuống hằng số thật sự.

Ba điều đáng mang theo. Thứ nhất, đây là một trong số ít trường hợp mà cận trên và cận dưới khớp nhau ở một hàm kỳ dị như α ; sự khớp đó là bằng chứng rằng hàm ấy thuộc về bài toán chứ không thuộc về cách ta phân tích. Thứ hai, và tổng quát hơn: mọi phát biểu “tối ưu” đều kèm một mô hình, và khi đọc một kết quả như vậy, câu hỏi đầu tiên phải là *tối ưu trong lớp thuật toán nào* (D/24). Thứ ba, về thực hành thì kết luận vẫn giản dị: đừng đi tìm DSU hằng số, và cũng đừng bỏ bớt một trong hai tối ưu — dùng riêng gộp theo kích thước hay riêng nén đường đều chỉ cho $O(\log n)$, phải có cả hai mới xuống tới α .

1.6 Gốc rễ: từ câu lệnh EQUIVALENCE của Fortran

Bài toán mà DSU giải ra đời cùng với những trình biên dịch đầu tiên. Ngôn ngữ Fortran có câu lệnh EQUIVALENCE cho phép lập trình viên khai báo rằng hai biến dùng chung một vùng nhớ. Trình biên dịch phải xử lý một loạt khai báo như vậy và tính ra các *lớp tương đương* — nhóm các biến thực sự chia sẻ bộ nhớ. Đó chính xác là bài toán “hợp nhất và hỏi cùng nhóm”.

Năm 1964, Bernard Galler và Michael Fischer công bố thuật toán cho bài toán đó, với cấu trúc rừng cây và ý tưởng đại diện mà ta dùng tới ngày nay. Điều đáng chú ý về mặt phương pháp: bài toán được phát hiện từ một nhu cầu công cụ rất cụ thể, giống hệt câu chuyện bảng băm ở chương B/08.

Phần lý thuyết mất thêm mười năm. Nén đường và gộp theo hạng được dùng khá sớm như những mẹo hiển nhiên, nhưng không ai chứng minh được chúng nhanh tới mức nào. Năm 1975 Tarjan cho lời giải: chi phí khấu hao là $O(\alpha(n))$ [1] — một kết quả gây bất ngờ vì hàm Ackermann ngược trước đó thuộc về logic và lý thuyết đệ quy, không ai chờ nó xuất hiện trong phân tích một cấu trúc dữ liệu thực dụng. Về sau người ta chứng minh thêm rằng cận này không cải thiện được, tức DSU với hai tối ưu là tối ưu về bản chất, không chỉ tối ưu trong các cách cài đã biết.

Còn một liên hệ ít được nhắc nhưng rất sâu: cùng cấu trúc này là hạt nhân của thuật toán *hợp nhất* trong suy diễn kiểu của trình biên dịch. Khi trình biên dịch suy ra kiểu của một biểu thức, nó liên tục phát biểu “kiểu này phải bằng kiểu kia” và phải duy trì các lớp tương đương của các biến kiểu — đúng bài toán của Galler và Fischer, chỉ khác ngữ cảnh. Nghĩa là DSU vẫn đang chạy trong trình biên dịch mỗi lần bạn dịch một chương trình có suy diễn kiểu.

I Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1964 — Galler & Fischer	Trình biên dịch Fortran phải xử lý câu lệnh EQUIVALENCE: các biến dùng chung vùng nhớ.	Cấu trúc rừng với đại diện; bài toán lớp tương đương được hình thức hoá.
Cuối thập niên 1960 — các cài đặt thực tế	Chuỗi cha dài làm thao tác tìm chậm.	Gộp theo hạng và nén đường xuất hiện như mẹo hiển nhiên, nhưng chưa ai biết chúng nhanh tới đâu.
1975 — Tarjan	Cần biết chính xác chi phí của hai mẹo đó.	Chi phí khấu hao $O(\alpha(n))$; hàm Ackermann ngược lần đầu xuất hiện trong phân tích một cấu trúc thực dụng.
Sau đó — các kết quả cận dưới	Có làm tốt hơn α được không?	Không: $O(\alpha(n))$ là tối ưu cho bài toán này trong mô hình con trỏ; một trong số ít cấu trúc dữ liệu đạt tối ưu chứng minh được.
Thập niên 1970–80 — suy diễn kiểu	Trình biên dịch phải duy trì các ràng buộc “kiểu này bằng kiểu kia”.	Thuật toán hợp nhất dùng đúng cấu trúc DSU; nó chạy mỗi lần bạn biên dịch một chương trình có suy diễn kiểu.

I 1.7 Liên hệ ngang

I Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Trình biên dịch	Hợp nhất trong suy diễn kiểu	Cùng cấu trúc, cùng thao tác: gộp hai lớp và hỏi cùng lớp. Điểm khác đáng chú ý: ở đó việc phát hiện mâu thuẫn (hai kiểu không hợp nhất được) chính là thông báo lỗi kiểu mà bạn nhìn thấy.
Thị giác máy tính	Gán nhãn thành phần liên thông trên ảnh nhị phân	Thuật toán hai lượt kinh điển dùng DSU để gộp các nhãn tạm thời. Ứng dụng gần: gộp các vùng ảnh hoặc các điểm đặc trưng thuộc cùng vật thể.
Vật lý thống kê	Mô phỏng thấm (percolation): các ô mở dần, hỏi khi nào hai bờ nối được với nhau	Đây là ứng dụng lịch sử nổi tiếng và trùng khít về hình thức: các ô mở dần chính là các phép gộp, câu hỏi “đã thấm chưa” chính là truy vấn cùng nhóm.
Mạng và hạ tầng	Kiểm tra liên thông mạng, thiết kế mạng chi phí nhỏ nhất	Kruskal (C/15) chạy trực tiếp trên DSU. Điểm khác trong thực tế: mạng thật có cả sự cố đứt kết nối, tức là có xoá — và đó chính là chỗ DSU thuận tuy không đủ.
Cơ sở dữ liệu, làm sạch dữ liệu	Gộp bản ghi trùng lặp thành một thực thể	“Hai bản ghi này là cùng một người” là một quan hệ tương đương được bổ sung dần — đúng mô hình DSU. Vấn đề thực tế: quyết định sai thì không rút lại được, đúng như hạn chế một chiều ở mục 2.
Toán học	Quan hệ tương đương và tập thương	DSU là hiện thực hoá tính toán của khái niệm tập thương: duy trì các lớp khi quan hệ sinh được bổ sung dần. Nhìn theo góc này thì “không tách được” là hệ quả tự nhiên: bỏ bớt một quan hệ sinh có thể làm cả cấu trúc lớp thay đổi.
SLAM	Gộp các quan sát về cùng một điểm mốc	Khi hai đường quan sát được xác định là cùng một điểm mốc thì hai tập quan sát phải gộp lại — đúng thao tác của DSU. Cùng cạm bẫy: nếu sau đó phát hiện gộp nhầm thì không tách ra được, nên trong thực tế người ta trì hoãn việc gộp cho tới khi đủ bằng chứng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì hai mẹo tấn công hai nguyên nhân khác nhau của chuỗi dài. Gộp theo kích thước ngăn không cho cây cao lên ngay từ đầu: một cây cao h buộc phải chứa ít nhất 2^h nút, nên chiều cao bị chặn bởi $\log_2 n$. Nén đường thì làm phẳng những đường đã đi qua, nên chi phí đã trả một lần không phải trả lại. Kết hợp lại, mọi cấu hình xấu đều tự triệt tiêu sau vài thao tác, và phân tích chặt cho ra $O(\alpha(n))$ khâu hao.
- A2. Vì nén đường đã phá huỷ thông tin về cấu trúc liên kết gốc: sau khi mọi nút trở thẳng lên gốc, không còn dấu vết của việc ai đã nối với ai. Đây là đánh đổi cố ý, và ba cách né là: xử lý ngược thời gian nếu bài chỉ có xoá; DSU có hoàn tác (bỏ nén đường, ghi nhật ký thao tác) kết hợp chia để trị trên trục thời gian nếu bài ngoại tuyến; hoặc dùng cấu trúc mạnh hơn như link-cut tree nếu buộc phải trực tuyến.
- A3. Mỗi khi một phần tử bị chuyển, nó đi từ tập nhỏ sang tập lớn, nên kích thước tập chứa nó ít nhất tăng gấp đôi. Kích thước không thể gấp đôi quá $\log_2 n$ lần trước khi vượt n , nên mỗi phần tử bị chuyển tối đa $\log_2 n$ lần, tổng $O(n \log n)$. Điều đáng chú ý về mặt phương pháp: ta không đếm chi phí theo từng thao tác gộp — có phép gộp rất đắt — mà đếm theo *tổng số lần các phần tử bị chạm*, đúng tinh thần phân tích khâu hao ở A/03.
- A4. Bằng cách lưu thêm một “khoảng cách” tương đối từ mỗi nút tới cha của nó, thường là bit chẵn lẻ. Khi đó “cùng phe” hay “khác phe” được suy ra từ tổng chẵn lẻ dọc đường lên gốc, và một ràng buộc mới mâu thuẫn với các ràng buộc cũ sẽ bị phát hiện ngay khi hai đầu đã cùng nhóm mà chẵn lẻ không khớp. Chỗ dễ sai nhất là phải cộng dồn trọng số trong lúc nén đường; quên bước đó thì cấu trúc vẫn chạy nhưng trả lời sai.
- A5. Vì DSU chỉ làm được một chiều — gộp — nên bài toán “xoá dần” là bài toán ngược của thứ nó làm được. Chạy các truy vấn theo thứ tự ngược thời gian biến mọi phép xoá thành phép thêm,

và lúc đó DSU dùng được nguyên vẹn. Cái giá là ta phải biết toàn bộ truy vấn từ trước, tức là chấp nhận lời giải ngoại tuyến — đây là cùng loại đánh đổi với thuật toán Mo ở B/10.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy hình dung một lớp học mà học sinh dần dần kết bạn. Bạn cần trả lời hai câu hỏi: “hai bạn này có chung một nhóm bạn không?” và “từ giờ hai nhóm này nhập làm một”.

Cách làm rất đơn giản: mỗi nhóm cử một bạn làm nhóm trưởng, và mỗi người chỉ cần nhớ mình theo ai. Muốn biết mình thuộc nhóm nào thì cứ hỏi ngược lên tới khi gặp người tự làm trưởng. Hai người cùng nhóm khi và chỉ khi họ ra cùng một trưởng.

Có hai mẹo làm việc này nhanh hẳn lên. Mẹo thứ nhất: khi nhập hai nhóm, luôn để nhóm đồng giữ trưởng và nhóm ít theo về — như vậy chuỗi hỏi ngược không bao giờ dài. Mẹo thứ hai: mỗi lần bạn phải hỏi ngược qua một chuỗi dài, hãy dặn tất cả những người trên đường “từ giờ cứ theo trưởng trưởng luôn” — lần sau ai hỏi cũng chỉ mất một bước. Hai mẹo tầm thường ấy làm chi phí gần như bằng hằng số, và điều đó phải mất mười năm mới được chứng minh chặt chẽ.

Nhưng có một điều DSU không làm được, và cần nhớ kỹ: *không tách nhóm ra được*. Sau khi mọi người đã theo trưởng trưởng, không ai còn nhớ ai đã kết bạn với ai lúc đầu, nên không có cách nào biết phải cắt ở đâu. Nếu bài toán của bạn có chuyện “huỷ kết bạn”, thì mẹo thường dùng là chạy ngược thời gian: xem cuốn phim từ cuối về đầu, lúc đó mọi việc huỷ trở thành việc kết.

Một câu: DSU chỉ hứa hai điều — cùng nhóm hay không, và gộp hai nhóm — và chính vì hứa ít nên nó gần như miễn phí.

Chỗ không rút gọn được: Chi phí $O(\alpha(n))$ thực sự không phải hằng số về mặt toán học; nó chỉ là hằng số về mặt thực tiễn vì hàm đó tăng quá chậm.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Hai thao tác của DSU	“Tìm đại diện” và “gộp hai nhóm”. Không lưu danh sách thành viên, không trả lời được “nhóm gồm những ai”.
••• Hai tối ưu và tác dụng	Gộp theo kích thước (cây cao h phải có $\geq 2^h$ nút $\Rightarrow$ chiều cao $\leq \log_2 n$); nén đường (làm phẳng đường vừa đi cho tương lai).
••• Chi phí khấu hao	$O(\alpha(n))$ với α là Ackermann ngược, thực tế ≤ 4 ; và cận này đã được chứng minh là tối ưu.
••• Hạn chế cốt lõi	Không xoá, không tách — nén đường đã phá huỷ cấu trúc liên kết gốc. Đây là đánh đổi cố ý, không phải thiếu sót.
••• Ba cách né hạn chế đó	Xử lý ngược thời gian (bài chỉ có xoá); DSU có hoàn tác + chia để trị trên trục thời gian (ngoại tuyến); link-cut tree (trực tuyến).
••• Gộp nhỏ vào lớn	Luôn đổ tập nhỏ vào tập lớn; mỗi phần tử bị chuyển $\leq \log_2 n$ lần vì mỗi lần chuyển thì kích thước tập chứa nó ít nhất gấp đôi. Tổng $O(n \log n)$.
•• DSU có trọng số / chặn lẻ	Lưu quan hệ tương đối tới cha; trả lời “cùng phe hay khác phe”, phát hiện mâu thuẫn. Phải cộng dồn trọng số khi nén đường.
•• Mẹo “vị trí trống kế tiếp”	Cho mỗi ô đã dùng trở sang ô trống kế tiếp; biến bài “tô đoạn, mỗi ô tô một lần” từ $O(n^2)$ thành gần tuyến tính.
•• Nén đường và hoàn tác	Không tương thích: muốn hoàn tác thì phải bỏ nén đường và chấp nhận $O(\log n)$.
•• DSU trên cây	Giữ cấu trúc đếm của con nặng nhất, gộp các con nhẹ vào; $O(n \log n)$ theo đúng lập luận gộp nhỏ vào lớn.
•• Kiểm tra đồ thị hai phía trực tuyến	DSU chặn lẻ: mâu thuẫn xuất hiện khi hai đầu một cạnh đã cùng nhóm mà cùng chặn lẻ.
• Gốc rễ	Ra đời 1964 cho câu lệnh EQUIVALENCE của Fortran; ngày nay vẫn chạy trong suy diễn kiểu của trình biên dịch.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●● “ α là tối ưu” nghĩa là gì	Tối ưu trong mô hình con trỏ (Tarjan, 1979) và vẫn $\Omega(\alpha)$ khẩu hao ngay trong mô hình cell-probe (Fredman–Saks, 1989). Mọi phát biểu tối ưu đều kèm một mô hình.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Chỉ dùng một trong hai tối ưu thì chi phí là bao nhiêu?

| Chỉ gộp theo kích thước cho $O(\log n)$ trường hợp xấu nhất — đây là đảm bảo thật, không phải khẩu hao, nên nó là lựa chọn khi cần hoàn tác. Chỉ nén đường cho $O(\log n)$ khẩu hao. Dùng cả hai cho $O(\alpha(n))$ khẩu hao. Trong thực hành, nếu không cần hoàn tác thì luôn dùng cả hai vì chúng chỉ tốn vài ký tự code.

2. Tôi có nên cài DSU bằng đệ quy không?

| Bản đệ quy cho hàm tìm đại diện rất ngắn và tự nhiên, nhưng có rủi ro tràn ngăn xếp nếu bạn chưa nén đường và cây sâu — tình huống này xảy ra trong các bài xây dựng đối nghịch. Bản lặp hai lượt (đi lên tìm gốc, đi lại lần nữa để nén) an toàn hơn và cũng nhanh hơn một chút. Với n lớn, nên dùng bản lặp; đây là ứng dụng cụ thể của lời khuyên ở E/25.

3. Làm sao dùng DSU để đếm số thành phần liên thông sau mỗi lần thêm cạnh?

| Khởi tạo số nhóm bằng n , và mỗi lần gộp thành công — tức hai đầu cạnh chưa cùng nhóm — thì giảm đi một. Cạnh nối hai đỉnh đã cùng nhóm không đổi gì, và đó cũng chính là dấu hiệu phát hiện chu trình, tức phần lõi của Kruskal ở C/15.

4. Khi nào tôi phải nén toạ độ trước khi dùng DSU?

| Khi các phần tử không phải số nguyên trong khoảng nhỏ. DSU cấp phát mảng theo số phần tử, nên với các khoá là số lớn, chuỗi hoặc cặp giá trị, bạn cần ánh xạ chúng về $0..n-1$ trước bằng cách sắp xếp và loại trùng, hoặc bằng bảng băm. Đây là bước tiền xử lý giống hệt như với Fenwick ở B/10.

5. Bài toán “liên thông động” đầy đủ — vừa thêm vừa xoá, trực tuyến — giải thế nào?

| Đây là bài toán khó thật sự, không phải bài tập thông thường. Với cây (không có chu trình), link-cut tree cho $O(\log n)$ khẩu hao. Với đồ thị tổng quát, các cấu trúc liên thông động có chi phí *polylogarit* nhưng cài đặt rất nặng. Trong thực hành thi đấu, nếu bài cho phép ngoại tuyến thì hãy dùng chia để trị trên trục thời gian với DSU có hoàn tác — ngắn hơn nhiều lần và đủ nhanh.

6. Gộp nhỏ vào lớn khác gì DSU thông thường?

| Khác ở chỗ nó giữ *dữ liệu thật* của mỗi nhóm — một tập hợp, một bảng đếm, một danh sách — chứ không chỉ giữ đại diện. Vì vậy nó tốn $O(n \log n)$ thay vì gần như tuyến tính, nhưng đổi lại trả lời được những câu mà DSU chịu, ví dụ “trong nhóm này có bao nhiêu giá trị phân biệt”. Quy tắc chọn: nếu chỉ cần biết cùng nhóm hay không thì DSU; nếu cần thống kê bên trong nhóm thì gộp nhỏ vào lớn.

7. Có bài nào trông không liên quan gì tới “nhóm” mà vẫn dùng DSU không?

| Có, và đó là dấu hiệu bạn đã thật sự nắm cấu trúc này. Mẹo “vị trí trống kế tiếp” ở mục 3 là ví dụ rõ nhất: bài toán nói về việc tô màu các ô, không nói gì về nhóm, nhưng quan hệ “ô này đã đầy, hãy hỏi ô bên phải” lại có đúng cấu trúc hợp nhất một chiều. Một ví dụ khác là các bài ràng buộc bằng nhau kiểu “ $x_i = x_j$ ”, nơi DSU dùng để phát hiện mâu thuẫn với các ràng buộc khác nhau.

I Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
• Đếm số thành phần liên thông khi thêm cạnh dần	CSES “Road Construction”[4]	DSU cơ bản kèm đếm kích thước nhóm lớn nhất.
• Kiểm tra đồ thị có chu trình khi thêm cạnh	Bài kinh điển	Cạnh nối hai đỉnh đã cùng nhóm chính là cạnh tạo chu trình.
•• Cây khung nhỏ nhất bằng Kruskal	CSES “Road Reparation”[4]	Sắp cạnh rồi DSU; chứng minh tính đúng ở $C/15$.
•• Kiểm tra đồ thị hai phía khi thêm cạnh dần	Bài kinh điển	DSU chặn lẻ; chú ý cộng dồn trọng số lúc nén đường.
•• Tô màu các đoạn, mỗi ô chỉ tô một lần	Bài thi đấu kinh điển	Mẹo “vị trí trống kế tiếp”; so với cây phân đoạn dễ thấy DSU ngắn hơn nhiều.
•• Số đảo trên lưới sau mỗi lần thêm đất	LeetCode “Number of Islands II”	DSU trên lưới; ánh xạ tọa độ hai chiều về chỉ số một chiều.
••• Liên thông khi các cạnh bị xoá dần	Bài kinh điển	Đảo ngược thời gian; đây là bài mẫu để hiểu mẹo đó.
••• Liên thông động ngoại tuyến với cạnh có khoảng thời gian tồn tại	Bài thi đấu kinh điển	DSU có hoàn tác + cây phân đoạn trên trục thời gian; kết hợp B/10 với chương này.
••• Với mỗi cây con, đếm số màu phân biệt	Bài kinh điển	DSU trên cây (gộp nhỏ vào lớn); tự chứng minh cận $O(n \log n)$ trước khi cài.
••• Ràng buộc “ x_i và x_j bằng nhau / khác nhau”, kiểm tra mâu thuẫn	Bài kinh điển	DSU chặn lẻ tổng quát; liên hệ với hợp nhất trong suy diễn kiểu ở mục 5.
••• Mô phỏng thăm trên lưới $n \times n$, tìm ngưỡng thăm	Bài tập thực nghiệm	Ứng dụng vật lý cổ điển; thêm hai nút ảo cho hai bờ là mẹo cài đặt cần thiết.

Kết chương và đọc tiếp

DSU là ví dụ rõ nhất trong quyển sách này cho luận điểm “hứa ít thì rẻ”: hai thao tác, một mảng số nguyên, và chi phí gần như hằng số với một chứng minh mà phải mất mười năm mới có. Hai điều đáng mang theo: lập luận gộp nhỏ vào lớn — mỗi phần tử chỉ bị chuyển $\log n$ lần — vì nó dùng lại ở nhiều nơi ngoài DSU; và ý thức về hạn chế một chiều, cùng ba cách né nó.

Chương cuối của phần B chuyển sang một miền dữ liệu riêng, nơi các cấu trúc chuyên biệt tạo ra khác biệt lớn nhất: chuỗi ký tự. Bạn sẽ thấy cùng một ý tưởng “đừng tính lại thứ đã biết” của chương B/06 xuất hiện lại dưới dạng hàm tiền tố của KMP, và sẽ thấy vì sao một cấu trúc như suffix array biến cả một họ bài chuỗi thành bài mảng thông thường (B/12).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Tarjan, “Efficiency of a Good But Not Linear Set Union Algorithm”	Journal of the ACM, 1975	Bài chứng minh cận $O(\alpha)$. Đọc phần phát biểu kết quả và định nghĩa α ; phần chứng minh dài và có thể để sau.
nền tảng	Tarjan, “A Class of Algorithms Which Require Nonlinear Time to Maintain Disjoint Sets”	Journal of Computer and System Sciences, 1979	Cận dưới trong mô hình con trỏ — tài liệu cho thấy “tối ưu” ở đây nghĩa là gì.
tra khi cần	Fredman & Saks, “The Cell Probe Complexity of Dynamic Data Structures”	STOC, 1989	Cận dưới $\Omega(\alpha)$ trong mô hình cell-probe. Đọc phần mở đầu; đây cũng là cửa vào cho các cận dưới cấu trúc dữ liệu nói chung.

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc (tiếp)
tra khi cần	Cormen, Leiserson, Rivest, Stein, <i>Introduction to Algorithms</i> (4th ed.), chương về tập rời nhau	MIT Press, 2022	Tra khi cần một chứng minh đầy đủ, viết theo chuẩn giáo trình, cho cận α .

Nguồn

- [1] R. E. Tarjan, “Efficiency of a Good But Not Linear Set Union Algorithm,” *Journal of the ACM*, vol. 22, no. 2, pp. 215–225, 1975.
- [2] R. E. Tarjan, “A Class of Algorithms Which Require Nonlinear Time to Maintain Disjoint Sets,” *Journal of Computer and System Sciences*, vol. 18, no. 2, 1979.
- [3] M. L. Fredman and M. E. Saks, “The Cell Probe Complexity of Dynamic Data Structures,” in *Proceedings of the 21st Annual ACM Symposium on Theory of Computing (STOC)*, 1989.
- [4] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).